

COMP1002

Final Assignment

Optimizing Urban Parcel Logistics Using Data Structures and Algorithms

Author: Yashika Jethwani

Student ID: 22519740

Submission Date: 28th July 2025

Table of Contents

1. Overview -----	3-4
2. Data Structures Used -----	5-6
3. Module Integration -----	7
4. Algorithm Complexity -----	8
5. Sample Output -----	9-16
6. Reflection -----	17
7. Limitations & Assumptions -----	17

Overview

1.1 Summary

As per the assignments requirement, I developed a comprehensive delivery management system that optimizes logistics operations through four integrated modules.

The four modules are:

- **Module 1 (Graph-Based Route Planning Optimization):** This module implements a complete graph structure using adjacency lists to represent delivery networks. The Graph class contains GraphNode objects that store connections with weights representing travel times.

I implemented three key algorithms:

- *BFS (Breadth-First Search):* This explores all reachable locations level by level, useful for determining service coverage areas.
- *DFS (Depth-First Search):* This uses recursive traversal to detect cycles in route networks, preventing infinite routing loops.
- *Dijkstra's Algorithm:* Calculates shortest paths.
- **Module 2 (Hash-Based Customer Lookup):** This contains 'dsaHashTable' class and this module provides fast customer data management. Each 'dsaHashEntry' stores customer information including ID, name, address, priority level (1-5), and delivery status.

The implementation features:

- *Hash Function:* Uses character sum modulo table size for key distribution.
- *Linear Probing:* Resolves collisions by checking the next available slot.
- *State Management:* Tracks entries as empty (0), active (1), or deleted (2) for efficient deletion without data shifting.
- *File Integration:* Loads customer data from CSV files and maintains it by updating it based on user operations like adding and deleting.

The system handles operations like searching for CUST001, deleting or adding new customers with their delivery details.

- **Module 3 (Heap-Based Parcel Scheduling):** The 'parcel_scheduling_heap' class implements a max-heap using the 'delivery_request_entry' structure.

This module manages delivery priorities through:

- *Priority Calculation:* Uses formula $(6 - \text{Priority_Level}) + (1000 / \text{Estimated_Time})$ to rank deliveries
- *Integration with Module 1:* Automatically calculates delivery times using Dijkstra's shortest path algorithm
- *Integration with Module 2:* Retrieves customer priority levels and status from the hash table
- *Heap Operations:* Maintains max-heap property through tree_up() and tree_down() methods
- *Address Mapping:* Converts customer addresses to graph locations for route planning

Only customers with "In Transit" status are scheduled, ensuring realistic delivery queue management. It also has option to ask user for the delivery time instead of automatically calculating it through Dijkstra.

- **Module 4 - Sorting Delivery Records:** In this module, the 'delivery_sorting_system' class provides comprehensive sorting analysis using 'delivery_record' objects.

In this module, I have implemented:

- **Merge Sort:** This guarantees $O(n \log n)$ performance with stable sorting.
- **Quick Sort:** Uses `partition_records()` with last element as pivot, providing efficient average-case performance
- **Performance Analysis:** Tracks comparisons, swaps, and execution time for algorithm comparison(random, nearly sorted, reversed).
- **Data Generation:** Creates random, nearly-sorted, and reverse-sorted datasets for testing,
- **Integration Support:** Loads delivery records from `customers.txt` and calculates estimated times using route planning

The system sorts records by estimated delivery time, enabling delivery optimization analysis.

1.2 Modular Design Approach

As asked in assignment, my code follows modular architecture principles where each module operates independently while supporting integration.

This has several benefits:

- **Maintainability:** Individual modules can be updated without affecting others.
- **Scalability:** New features can be added to specific modules as needed.
- **Reusability:** Modules can be imported and used in other applications.
- **Testing:** Each module can be tested and debugged separately.

Data Structures Used

2.1 Module 1

Implementation: Adjacency list using Python dictionaries and lists and BFS, DFS, Dijkstra(shortest path calculator algorithm)

Technical Justification:

- **Space Efficiency:** $O(V + E)$ space complexity, which is optimal for typical delivery route networks.
- **Traversal Performance:** Enables efficient BFS and DFS operations for connectivity analysis.
- **Weighted Edges:** Supports travel time calculations for route optimization.

Key Technical Features:

- Undirected graph implementation where `add_edge()` creates bidirectional connections automatically.
- Error handling including validation for negative weights, None nodes, and self-loops.
- Dynamic node creation through `add_node()` method.
- Custom queue operations with `dequeue()` implementation using array shifting.
- Utility methods like `sorting_keys()` using bubble sort and `membership_check()` for list operations.

2.2 Module 2

Implementation: Open addressing with linear probing collision resolution.

Technical Justification:

- **Access Performance:** Average $O(1)$ time complexity for search, insert, and delete operations.
- **Memory Efficiency:** Single array storage with minimal overhead compared to chaining.
- **Cache Performance:** Linear probing provides better spatial locality than separate chaining.
- **Use Case Fit:** Ideal for frequent customer lookups in delivery operations.

Key Technical Features:

- Three-state entry management (empty, active, deleted).
- Linear probing collision resolution in `insert_customer()` with step counting for debugging.
- Deletion marking entries as deleted rather than physically removing them.
- Load factor tracking through `self.count` for performance monitoring.
- File integration with `load_file()` parsing CSV format: `customer_id,name,address,Priority Level,status`

2.3 Module 3

Implementation: Array-based binary heap with priority queue operations.

Technical Justification:

- **Priority Operations:** $O(\log n)$ insertion and extraction maintaining efficient queue operations.
- **Memory Layout:** Array representation with implicit parent-child relationships.
- **Heap Property:** Max-heap ensures highest priority deliveries are always accessible.
- **Dynamic Maintenance:** Trickle-up and trickle-down operations maintain heap invariants

Key Technical Features:

- Priority calculation using formula $(6 - \text{customer_priority}) + (1000.0 / \text{estimated_time})$ in `insert()` method.
- Integration with Module 1 through `route_graph.dijkstra(warehouse)` for automatic time calculation.
- Integration with Module 2 via `customer_table.search_customer(customer_id)` for priority level retrieval.
- Heap maintenance through `tree_up()` bubbling elements upward and `tree_down()` shifting elements downward.
- Address mapping system `address_to_route_location()` converting customer addresses to graph nodes.
- Ensuring only customers with `status.strip().lower() == 'in transit'` are scheduled.

2.4 Module 4

Implementation: Both merge sort and quick sort with performance analysis

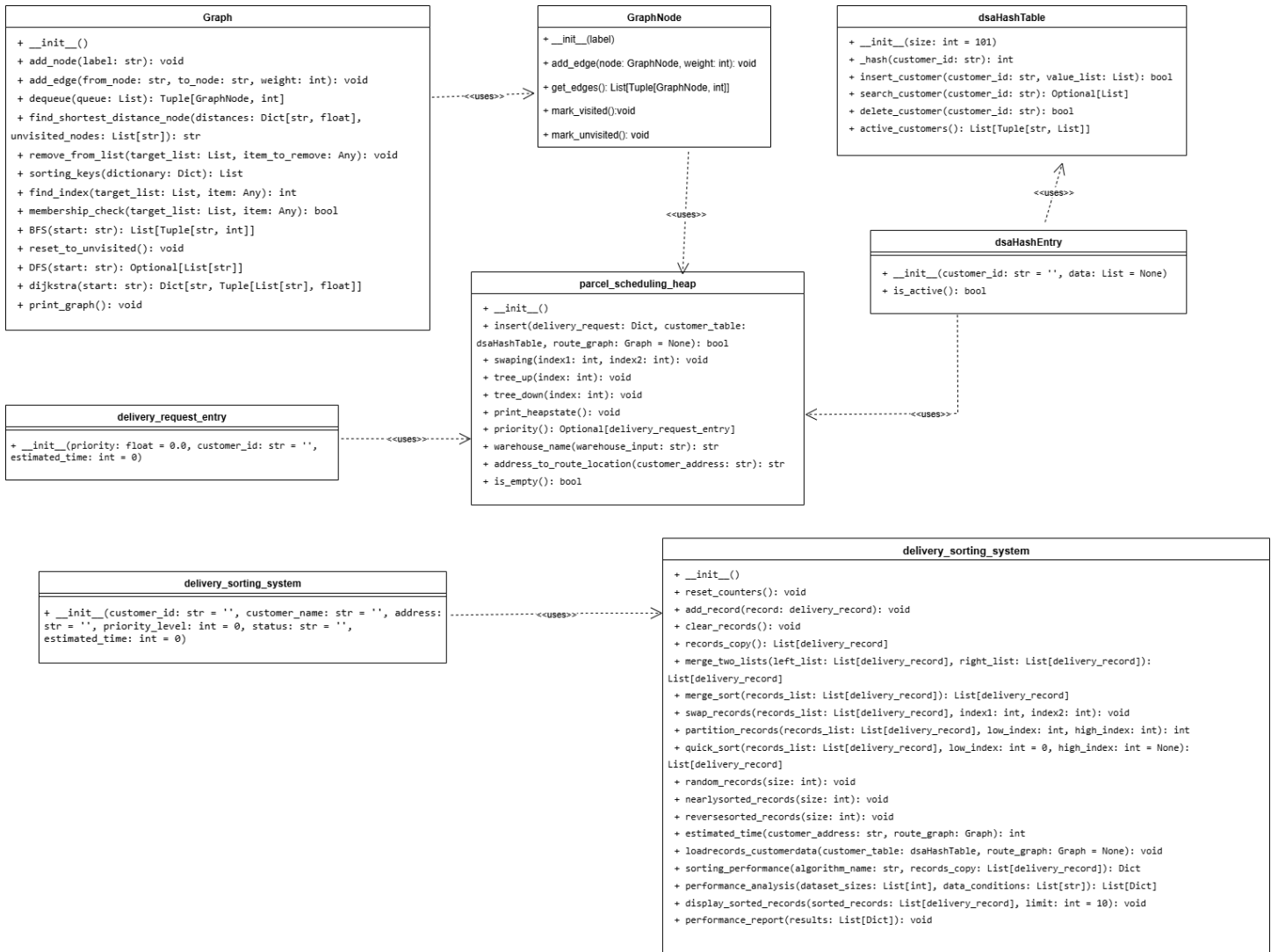
Technical Justification:

- **Merge Sort:** Guarantees $O(n \log n)$ performance with stable sorting behavior
- **Quick Sort:** Provides $O(n \log n)$ average case with better practical performance
- **Comparative Analysis:** Enables performance evaluation across different data distributions
- **Scalability:** Both algorithms handle large datasets efficiently

Key Technical Features:

- Custom comparison logic based on estimated delivery times.
- Performance analysis (comparisons, swaps, execution time)
- Support for different data conditions (random, nearly sorted, reverse sorted)

Module Integration



Algorithm Complexity

4.1 Module 1

Algorithm	Time Complexity	Space Complexity	Primary Use Case
BFS	$O(V + E)$	$O(V)$	Reachability analysis from warehouse
DFS	$O(V + E)$	$O(V)$	Cycle detection in route networks
Dijkstra	$O(V^2)$	$O(V)$	Shortest path calculation

Note: In my code, Dijkstra implementation uses linear search for minimum distance node.

A priority queue implementation would improve this to $O((V + E) \log V)$.

4.2 Module 2

Operation	Average Case	Worst Case	Space Complexity
Insert	$O(1)$	$O(n)$	$O(n)$
Search	$O(1)$	$O(n)$	$O(1)$
Delete	$O(1)$	$O(n)$	$O(1)$

- Load factor management prevents clustering.
- Linear probing provides good cache locality.
- Worst case occurs when all the keys hash to consecutive positions.

4.3 Module 3

Operation	Time Complexity	Space Complexity	Description
Insert	$O(\log n)$	$O(1)$	Add delivery with priority calculation
Extract Max	$O(\log n)$	$O(1)$	Remove highest priority delivery
Trickle Up 'tree_up()'	$O(\log n)$	$O(1)$	Maintain heap property after insertion
Trickle Down 'tree_down()'	$O(\log n)$	$O(1)$	Restore heap property after extraction

4.4 Module 4

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n \log n)$

Sample Output

My code can be run on VS Code, Windows Terminal and Linux Terminal.

I will be providing sample outputs for each module by running it on Linux Terminal.

To run:

python3 filename.py

5.1 Module 1

```

yashika@Yashika: /mnt/c/Users/abc47/OneDrive/Desktop/dsa$ python3 Module1.py
Graph Structure:
A -> ['B(5)', 'C(3)']
B -> ['A(5)', 'D(2)']
C -> ['A(3)', 'D(4)', 'E(7)']
D -> ['B(2)', 'C(4)', 'F(1)']
E -> ['C(7)', 'F(3)', 'G(6)']
F -> ['D(1)', 'E(3)', 'G(2)']
G -> ['E(6)', 'F(2)']
H -> []

BFS from A:
Node A: Level 0
Node B: Level 1
Node C: Level 1
Node D: Level 2
Node E: Level 2
Node F: Level 3
Node G: Level 3

DFS Cycle Detection:
Cycle found: A -> B -> D -> C -> A

Dijkstra's Shortest Paths from A:
To A: A, Cost: 0
To B: A -> B, Cost: 5
To C: A -> C, Cost: 3
To D: A -> C -> D, Cost: 7
To E: A -> C -> E, Cost: 10
To F: A -> C -> D -> F, Cost: 8
To G: A -> C -> D -> F -> G, Cost: 10
To H: No path available
yashika@Yashika: /mnt/c/Users/abc47/OneDrive/Desktop/dsa$

```

5.2 Module 2

Checking the index and performing linear probing for each customer as soon as you enter the dataset file name. Ex – customers.txt

```

yashika@Yashika:/mnt/c/Users/abc47/OneDrive/Desktop/dsa$ python3 Module2.py
CUSTOMER LOOKUP SYSTEM
Enter customer file name (e.g., customers.txt): customers.txt
Inserting key='CUST000' - Initial idx=59 - Inserted at index 59, step=0
Inserting key='CUST001' - Initial idx=60 - Inserted at index 60, step=0
Inserting key='CUST002' - Initial idx=61 - Inserted at index 61, step=0
Inserting key='CUST003' - Initial idx=62 - Inserted at index 62, step=0
Inserting key='CUST004' - Initial idx=63 - Inserted at index 63, step=0
Inserting key='CUST005' - Initial idx=64 - Inserted at index 64, step=0
Inserting key='CUST006' - Initial idx=65 - Inserted at index 65, step=0
Inserting key='CUST007' - Initial idx=66 - Inserted at index 66, step=0
Inserting key='CUST008' - Initial idx=67 - Inserted at index 67, step=0
Inserting key='CUST009' - Initial idx=68 - Inserted at index 68, step=0
Inserting key='CUST010' - Initial idx=60
Collision at index 60, probing...

Collision at index 61, probing...

Collision at index 62, probing...

Collision at index 63, probing...

Collision at index 64, probing...

Collision at index 65, probing...

Collision at index 66, probing...

Collision at index 67, probing...

Collision at index 68, probing...
- Inserted at index 69, step=9
Inserting key='CUST011' - Initial idx=61
Collision at index 61, probing...

Collision at index 62, probing...

Collision at index 63, probing...

Collision at index 64, probing...

Collision at index 65, probing...

```

Opting for option 1 – Search

Case 1 – Customer ID exists

```

Menu:
1. Search customer
2. Delete customer
3. Add new customer
4. Exit
Enter your choice (1/2/3/4): 1
Enter Customer ID to search: CUST022
Customer 'CUST022' found:
  Name      : Diya Kapoor
  Address   : 111 Maple Road
  Priority Level: 5
  Status    : In Transit

```

Case 2 – Customer ID does not exists

```
Menu:
1. Search customer
2. Delete customer
3. Add new customer
4. Exit
Enter your choice (1/2/3/4): 1
Enter Customer ID to search: CUST088
Customer ID not found.
```

Opting for option 2 – Delete

```
Menu:
1. Search customer
2. Delete customer
3. Add new customer
4. Exit
Enter your choice (1/2/3/4): 2
Enter Customer ID to delete: CUST052
Deleted successfully and file updated.
```

Opting for option 3 – Add new

```
Menu:
1. Search customer
2. Delete customer
3. Add new customer
4. Exit
Enter your choice (1/2/3/4): 3
Enter Customer ID: CUST052
Enter Name: Yashika
Enter Address: BurDubai
Enter Priority Level (1-5): 1
Enter Status (e.g., Delivered/In Transit/Delayed): Delayed
Inserting key='CUST052' - Initial idx=66
Collision at index 66, probing...

Collision at index 67, probing...

Collision at index 68, probing...

Collision at index 69, probing...

Collision at index 70, probing...
```

```
Collision at index 8, probing...

Collision at index 9, probing...
- Inserted at index 10, step=45
Customer details added and file updated successfully.
```

5.3 Module 3

```
yashika@Yashika:/mnt/c/Users/abc47/OneDrive/Desktop/dsa$ python3 Module3.py
PARCEL SCHEDULING SYSTEM
Enter customer data filename (e.g., customers.txt): customers.txt
Inserting key='CUST042' - Initial idx=65 - Inserted at index 65, step=0
Inserting key='CUST043' - Initial idx=66 - Inserted at index 66, step=0
Inserting key='CUST044' - Initial idx=67 - Inserted at index 67, step=0
Inserting key='CUST045' - Initial idx=68 - Inserted at index 68, step=0
Inserting key='CUST046' - Initial idx=69 - Inserted at index 69, step=0
Inserting key='CUST047' - Initial idx=70 - Inserted at index 70, step=0
Inserting key='CUST048' - Initial idx=71 - Inserted at index 71, step=0
Inserting key='CUST049' - Initial idx=72 - Inserted at index 72, step=0
Inserting key='CUST050' - Initial idx=64 - Inserted at index 64, step=0
Inserting key='CUST051' - Initial idx=65
Collision at index 65, probing...

Collision at index 66, probing...

Collision at index 67, probing...
```

Case 1 – load and use routes data 'routes.txt'

```
Collision at index 7, probing...

Collision at index 8, probing...
- Inserted at index 9, step=46
Customer data loaded successfully from customers.txt
Do you want to use Module 1 route planning? (y/n): y
Enter route data filename (e.g., routes.txt): routes.txt
Loading route data from routes.txt...
Added route: WarehouseA ↔ DistributionHub (10 min)
Added route: WarehouseB ↔ DistributionHub (15 min)
Added route: DistributionHub ↔ CentralStation (8 min)
Added route: WarehouseA ↔ Palm Street (20 min)
Added route: WarehouseA ↔ Sunset Blvd (25 min)
Added route: WarehouseA ↔ Creek Ave (22 min)
Added route: WarehouseA ↔ Maple Road (28 min)
Added route: WarehouseA ↔ Garden Lane (30 min)
```

1. Add delivery request

(We have two ways to calculate)

a. Route planning

```
Witrow Way > [ Forest Path42 ]
Ready!

=== MENU ===
1. Add delivery request
2. Process next delivery (extract highest priority)
3. View heap status
4. Process all deliveries
5. Exit
Enter your choice (1-5): 1

ENTER DELIVERY REQUEST
Enter Customer ID: CUST012

Choose input method:
1. Use Module 1 (Route Planning) - Auto-calculate from warehouse to customer address
2. Manual input - Enter estimated time directly
Enter choice (1/2): 1

INSERTING DELIVERY REQUEST
Customer ID: CUST012
Destination:
Mapped customer address '231 Maple Road' to destination: Maple Road
Calculating route from WarehouseA to Maple Road using Module 1...
Shortest path found: WarehouseA → Maple Road
Estimated time from Module 1 Dijkstra: 28 minutes
Customer Priority Level: 3
Customer Status: In Transit
Priority Calculation: (6 - 3) + (1000 / 28) = 38.71
Successfully inserted: CUST012 with priority 38.71

--- HEAP STATE (Size: 1) ---
Index 0: Customer CUST012 - Priority: 38.71 - Time: 28min
END HEAP STATE
```

b. Manual time insertion by user

```

=== MENU ===
1. Add delivery request
2. Process next delivery (extract highest priority)
3. View heap status
4. Process all deliveries
5. Exit
Enter your choice (1-5): 1

ENTER DELIVERY REQUEST
Enter Customer ID: CUST012

Choose input method:
1. Use Module 1 (Route Planning) - Auto-calculate from warehouse to customer address
2. Manual input - Enter estimated time directly
Enter choice (1/2): 2
Enter Estimated Delivery Time (minutes): 20

INSERTING DELIVERY REQUEST
Customer ID: CUST012
Destination:
Using provided estimated time by the user: 20 minutes
Customer Priority Level: 3
Customer Status: In Transit
Priority Calculation:  $(6 - 3) + (1000 / 20) = 53.00$ 
Successfully inserted: CUST012 with priority 53.00

--- HEAP STATE (Size: 1) ---
Index 0: Customer CUST012 - Priority: 53.00 - Time: 20min
END HEAP STATE

```

2. Process next delivery

```

=== MENU ===
1. Add delivery request
2. Process next delivery (extract highest priority)
3. View heap status
4. Process all deliveries
5. Exit
Enter your choice (1-5): 2

EXTRACTING HIGHEST PRIORITY DELIVERY
Extracting: Customer CUST012
Priority: 38.71
Estimated Time: 28 minutes
Successfully extracted: CUST012

--- HEAP STATE (Size: 0) ---
Heap is empty
END HEAP STATE

Next delivery: Customer CUST012

```

3. Heap status

```

=== MENU ===
1. Add delivery request
2. Process next delivery (extract highest priority)
3. View heap status
4. Process all deliveries
5. Exit
Enter your choice (1-5): 3

--- HEAP STATE (Size: 0) ---
Heap is empty
END HEAP STATE

```

4. Process all deliveries


```

=== MENU ===
1. Add delivery request
2. Process next delivery (extract highest priority)
3. View heap status
4. Process all deliveries
5. Exit
Enter your choice (1-5): 4

PROCESSING ALL DELIVERIES IN PRIORITY ORDER

EXTRACTING HIGHEST PRIORITY DELIVERY
Extracting: Customer CUST008
Priority: 47.45
Estimated Time: 22 minutes
Successfully extracted: CUST008

--- HEAP STATE (Size: 0) ---
Heap is empty
END HEAP STATE

Delivery 1: Customer CUST008
Processed 1 deliveries successfully

```

Case 2 – not use routes data 'routes.txt'

```

Collision at index 8, probing...
- Inserted at index 9, step=46
Customer data loaded successfully from customers.txt
Do you want to use Module 1 route planning? (y/n): n
Using manual time input mode only.

=== MENU ===
1. Add delivery request
2. Process next delivery (extract highest priority)
3. View heap status
4. Process all deliveries
5. Exit
Enter your choice (1-5): 1

ENTER DELIVERY REQUEST
Enter Customer ID: CUST008

Choose input method:
1. Use Module 1 (Route Planning) - Auto-calculate from warehouse to customer address
2. Manual input - Enter estimated time directly
Enter choice (1/2): 2
Enter Estimated Delivery Time (minutes): 30

INSERTING DELIVERY REQUEST
Customer ID: CUST008
Destination:
Using provided estimated time by the user: 30 minutes
Customer Priority Level: 4
Customer Status: In Transit
Priority Calculation:  $(6 - 4) + (1000 / 30) = 35.33$ 
Successfully inserted: CUST008 with priority 35.33

--- HEAP STATE (Size: 1) ---
Index 0: Customer CUST008 - Priority: 35.33 - Time: 30min
END HEAP STATE

```

5.4 Module 4

1. Load data from file to perform merge and quick sort

```

yashika@Yashika: /mnt/c/Users/abc47/OneDrive/Desktop/dsa$ python3 Module4.py
1. Load data from file
2. Performance analysis
3. Exit
Choice: 1
Customer file: customers.txt
Inserting key='CUST042' - Initial idx=65 - Inserted at index 65, step=0
Inserting key='CUST043' - Initial idx=66 - Inserted at index 66, step=0
Inserting key='CUST044' - Initial idx=67 - Inserted at index 67, step=0
Inserting key='CUST045' - Initial idx=68 - Inserted at index 68, step=0
Inserting key='CUST046' - Initial idx=69 - Inserted at index 69, step=0
Inserting key='CUST047' - Initial idx=70 - Inserted at index 70, step=0
Inserting key='CUST048' - Initial idx=71 - Inserted at index 71, step=0
Inserting key='CUST049' - Initial idx=72 - Inserted at index 72, step=0
Inserting key='CUST050' - Initial idx=64 - Inserted at index 64, step=0
Inserting key='CUST051' - Initial idx=65
Collision at index 65, probing...

Collision at index 66, probing...

Collision at index 67, probing...

Collision at index 68, probing...

```

We have option to load the routes.txt data if we want, if not then not necessary. Also we can choose to display the sorted data or not.

So let's make 2 cases

Case 1- Merge sort with routes.txt with displaying the data.

```

Loaded customer data from customers.txt
Routes file (optional): routes.txt
Loaded route data from routes.txt
Loaded 52 delivery records from customer data
1. Merge Sort
2. Quick Sort
Algorithm: 1

merge - Time: 0.17ms
Comparisons: 221, Swaps: 0
Show records? (y/n): y

Sorted records (by estimated time):
CUST032: 18min
CUST033: 18min
CUST034: 18min
CUST036: 18min
CUST038: 18min
CUST039: 18min
CUST040: 18min
CUST041: 18min
CUST042: 18min
CUST043: 18min
... (52 total)

```

Case 2 – Quick sort without routes.txt

```

Loaded customer data from customers.txt
Routes file (optional): → Just press enter
Loaded 52 delivery records from customer data
1. Merge Sort
2. Quick Sort
Algorithm: 2

quick - Time: 0.12ms
Comparisons: 262, Swaps: 95
Show records? (y/n): n

```

2. Performance Analysis

```

1. Load data from file
2. Performance analysis
3. Exit
Choice: 2
Running performance analysis...
Testing dataset size: 100
  random data condition...
Generated 100 random delivery records
  merge sort: 0.24ms
  quick sort: 0.14ms
  nearly_sorted data condition...
Generated 100 random delivery records
Generated 100 nearly sorted delivery records
  merge sort: 0.20ms
  quick sort: 0.55ms
  reverse_sorted data condition...
Generated 100 random delivery records
Generated 100 reverse sorted delivery records

```

Performance Report

```

-----
Size: 100
random: Merge 0.24ms, Quick 0.14ms
nearly_sorted: Merge 0.20ms, Quick 0.55ms
reverse_sorted: Merge 0.25ms, Quick 0.37ms

```

```

Size: 500
random: Merge 1.39ms, Quick 2.00ms
nearly_sorted: Merge 1.23ms, Quick 1.69ms
reverse_sorted: Merge 1.16ms, Quick 7.84ms

```

```

Size: 1000
random: Merge 2.94ms, Quick 2.84ms
nearly_sorted: Merge 6.13ms, Quick 8.10ms
reverse_sorted: Merge 4.41ms, Quick 40.57ms

```

```

Analysis:
Merge Sort:  $O(n \log n)$  consistent
Quick Sort:  $O(n \log n)$  average,  $O(n^2)$  worst case

```


Reflection

This project helped me understand how data structures actually work. Building this system taught me to choose the right algorithm - we have to consider what data actually looks like and how the system will be used in the real world.

Working with graphs, hash tables, and heaps gave me hands-on experience with their performance characteristics. I learned that sometimes a theoretically slower algorithm can actually perform better with certain types of input data. Dividing the whole program in different modules was very helpful - it forced me to think carefully about how different components should interact without being too tightly coupled.

Limitations & Assumptions

Technical Limitations

The current implementation has some notable restrictions that I'm aware of. The graph module uses a basic $O(V^2)$ implementation of Dijkstra's algorithm instead of the more efficient heap-based version. The hash table uses linear probing which can create clustering problems, and it does not automatically resize when it's full.

Also, if I need to update the delivery's priority after it's in the heap, it needs to be removed and then add it back.

System Assumptions

- Customer data is clean and properly formatted.
- Route information is complete and accurate.
- The system has enough memory to handle the expected data size.
- All deliveries start from one central warehouse.
- Travel times don't change based on traffic or time of day.